BÁO CÁO THỰC HÀNH LAB 01

SQL trên Google Cloud Shell với SQLite

Môn: Introduction to Data Science · PTIT | Họ tên: ..... MSSV: ..... Lớp: .....

Ngày thực hiện: 2026-11-05 | Python 3.11 · SQLite 3.40 · ảnh minh chứng sinh tự động từ kết quả chạy thật

Tóm tắt. Xây dựng cơ sở dữ liệu quan hệ bán hàng trực tuyến bốn bảng (customers, products, orders, order_items) trên SQLite trong Google Cloud Shell; thực hiện CRUD, JOIN, GROUP BY/HAVING, subquery, window function; tạo index và đọc EXPLAIN QUERY PLAN để chứng minh ảnh hưởng của index tới kế hoạch thực thi. Cả 21 yêu cầu S01–S21 đã cài đặt và chạy kiểm chứng: 0 lỗi, khoá ngoại hợp lệ, kế hoạch chuyển từ SCAN orders sang SEARCH orders USING COVERING INDEX sau khi tạo index.

1. Môi trường và cách tổ chức bài làm

Thành phần	Giá trị
Nền tảng	Google Cloud Shell, chế độ $HOME bền vững (không bật Billing)
Python / CSDL	Python 3.11.2 · thư viện chuẩn sqlite3 · SQLite 3.40.1
Công cụ trong bài	sql_console.py, build_db.py, generate_data.py, run_sql.py
Chi phí	0 đồng — chỉ dùng tài nguyên miễn phí

Tổ chức. Đề có hai khối yêu cầu với điều kiện dữ liệu khác nhau nên bài làm tách hai tệp đáp án riêng: phần BẮT BUỘC S01–S15 trong sql/answers.sql chạy trên data/sales.db (dữ liệu mẫu của đề); phần BỔ SUNG/về nhà S16–S21 trong sql/answers_homework.sql chạy trên data/sales_extended.db (dữ liệu mở rộng); phần chứng minh index ở sql/explain.sql.

2. Mô hình dữ liệu

```
CREATE TABLE customers (customer_id INTEGER PRIMARY KEY, name TEXT NOT NULL, city TEXT);
CREATE TABLE products  (product_id INTEGER PRIMARY KEY, name TEXT NOT NULL,
                        category TEXT, price REAL CHECK(price >= 0));
CREATE TABLE orders    (order_id INTEGER PRIMARY KEY, customer_id INTEGER NOT NULL,
                        order_date TEXT,
                        FOREIGN KEY(customer_id) REFERENCES customers(customer_id));
CREATE TABLE order_items (order_id INTEGER, product_id INTEGER,
                        quantity INTEGER CHECK(quantity > 0),
                        PRIMARY KEY(order_id, product_id),
                        FOREIGN KEY(order_id)   REFERENCES orders(order_id),
                        FOREIGN KEY(product_id) REFERENCES products(product_id));
```

- Khoá chính tổ hợp (order_id, product_id) ngăn một sản phẩm xuất hiện hai lần trong cùng đơn — muốn mua thêm thì tăng quantity.
- CHECK chặn giá âm và số lượng ≤ 0 ngay tại tầng CSDL, không phụ thuộc ứng dụng.
- SQLite KHÔNG kiểm tra khoá ngoại mặc định, nên mọi kết nối trong bài đều chạy PRAGMA foreign_keys = ON ngay sau khi mở CSDL — thiếu bước này ràng buộc chỉ là chú thích.



Hình 1 — Nạp dữ liệu mẫu: 5 khách · 5 sản phẩm · 6 đơn · 8 dòng đơn, PRAGMA foreign_key_check rỗng

3. Kết quả phần bắt buộc (S01–S15)

Chạy python3 run_sql.py sql/answers.sql thực thi 18 câu lệnh với 0 lỗi; chạy lần thứ hai vẫn 0 lỗi và cho kết quả giống hệt. Bảng dưới tóm tắt kỹ thuật và số dòng thực tế của từng câu.

Mã	Kỹ thuật	Dòng	Mã	Kỹ thuật	Dòng
S01	SELECT + ORDER BY	5	S09	LEFT JOIN + GROUP BY	5
S02	WHERE price > 1.000.000	2	S10	HAVING SUM(...) > 5tr	3
S03	ORDER BY price DESC	5	S11	subquery AVG(price)	1
S04	INSERT ... ON CONFLICT	1	S12	HAVING COUNT(đơn) ≥ 2	1
S05	UPDATE price = price*1.1	1	S13	ORDER BY + LIMIT 3	3
S06	DELETE + NOT EXISTS	1	S14	CREATE INDEX	—
S07	INNER JOIN 2 bảng	6	S15	EXPLAIN QUERY PLAN	1
S08	JOIN 3 bảng + SUM	6	—		



Hình 2 — S08 tổng tiền từng đơn và S09 doanh thu từng khách (giá Mouse đã tăng 10% ở S05)

CRUD lặp an toàn. Ba câu có ghi (S04–S06) được viết để file chạy lại không sai dữ liệu: INSERT có ON CONFLICT DO NOTHING; UPDATE kèm điều kiện AND price = 250000 nên lần chạy thứ hai trả về Rows affected: 0 (giá vẫn 275.000, không cộng dồn 10%); DELETE kèm NOT EXISTS đúng nghĩa "khách vừa thêm chưa có đơn" và bảo vệ khoá ngoại. S10 lọc theo kết quả tổng hợp nên dùng HAVING (3 khách vượt 5 triệu); S11 so với giá trung bình 4.045.000 nên chỉ Laptop vượt; S12 đúng 1 khách (An, 2 đơn); S13 top 3 là Laptop 30.000.000, Monitor 10.500.000, Headphones 1.600.000 — LIMIT không xử lý đồng hạng nên muốn đúng phải dùng RANK() như S19.

4. Index và kế hoạch thực thi (S14–S15)

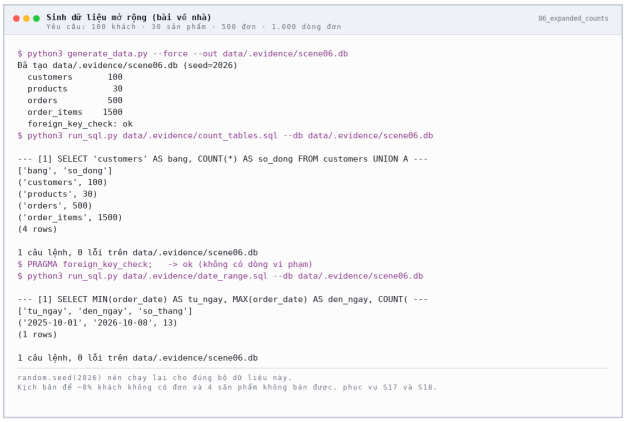
explain.sql tự DROP rồi CREATE index nên luôn đo được cả hai trạng thái: DROP INDEX IF EXISTS → EXPLAIN → CREATE INDEX IF NOT EXISTS → EXPLAIN.



Hình 3 — Trước index: SCAN orders; sau index: SEARCH orders USING COVERING INDEX idx_orders_customer

- SCAN đọc tuần tự toàn bảng rồi mới lọc — chi phí tăng tuyến tính theo số dòng.
- SEARCH đi xuống cây B-tree của index — chi phí ~log n cộng số dòng khớp.
- COVERING INDEX: câu chỉ cần order_id, mà index trên customer_id của bảng có INTEGER PRIMARY KEY đã chứa rowid (chính là order_id), nên SQLite trả lời hoàn toàn từ index; nếu lấy thêm order_date (S20) thì kế hoạch là SEARCH ... USING INDEX.
- Không kết luận bằng thời gian chạy: với 6 đơn, chênh lệch nằm dưới ngưỡng đo của đồng hồ hệ thống — phải đọc plan (kiểu truy cập, có phải sort thêm hay không).

5. Kết quả phần bổ sung (S16–S21)



Hình 4 — Dữ liệu mở rộng: 100 khách · 30 sản phẩm · 500 đơn · 1.500 dòng đơn, khoá ngoại ok, trải 13 tháng

S16 Doanh thu theo tháng (strftime('%Y-%m')): 13 nhóm tháng 10/2025–10/2026; cao nhất 2025-10 với 1.142.485.000 đ / 53 đơn; tháng cuối chỉ 7 đơn (tháng dở dang).

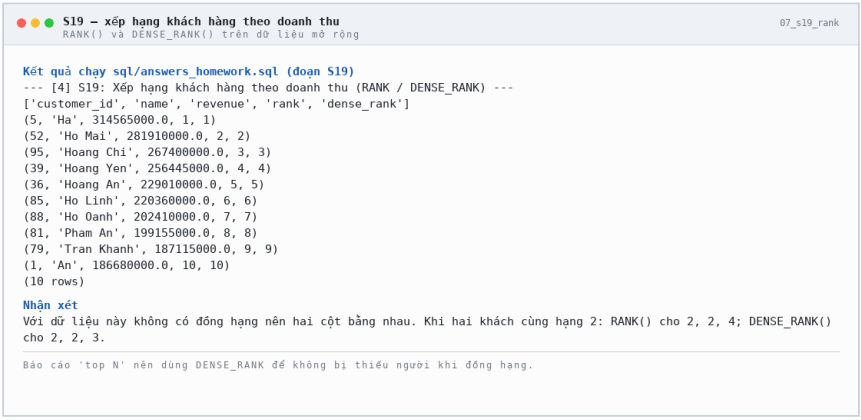
S17 Khách chưa có đơn (LEFT JOIN ... IS NULL): 8 khách — id 37, 38, 43, 56, 83, 89, 92, 94.

S18 Sản phẩm chưa từng bán (NOT EXISTS): 4 sản phẩm — Graphics Card, RAM 16GB, CPU Cooler, Power Supply.

S19 Xếp hạng bằng window function: top 3 Ha 314.565.000 · Ho Mai 281.910.000 · Hoang Chi 267.400.000. Dữ liệu không đồng hạng nên RANK = DENSE_RANK; khi đồng hạng, RANK cho 2, 2, 4 còn DENSE_RANK cho 2, 2, 3.

S20 Sau khi mở rộng: chưa index là SCAN orders, có index là SEARCH orders USING INDEX kèm USE TEMP B-TREE FOR ORDER BY (do ORDER BY order_date). Nếu truy vấn chạy thường xuyên, index tổ hợp (customer_id, order_date) sẽ bỏ được bước sort.

S21 SQL Server: #t là bảng tạm cục bộ theo session, ##t là bảng tạm toàn cục dùng chung giữa các session, cả hai sống trong tempdb (CSDL hệ thống, chứa cả biến bảng và sort/hash spill). SQLite chỉ có CREATE TEMP TABLE gắn với kết nối, nằm trong schema temp (RAM hoặc tệp tạm tuỳ PRAGMA temp_store) và KHÔNG có bảng tạm toàn cục; bảng temp được tìm trước main nên trùng tên sẽ che bảng thường.



Hình 5 — S19 xếp hạng khách hàng theo doanh thu bằng RANK / DENSE_RANK

6. Kết luận và bài học

Cả 21 yêu cầu đã hoàn thành. Bốn bài học: (i) khoá ngoại SQLite là tuỳ chọn — phải bật pragma; (ii) script có ghi phải lặp an toàn (ON CONFLICT, điều kiện chặt trong UPDATE, NOT EXISTS trong DELETE); (iii) index không miễn phí — chỉ tạo cho cột thường lọc/nối; (iv) đọc plan thay vì đo thời gian, chú ý COVERING INDEX và USE TEMP B-TREE FOR ORDER BY vì chúng chỉ ra chi phí ẩn.



Hình 6 — SQL Console: câu lệnh nhiều dòng, thông báo lỗi và tiếp tục nhận câu lệnh mới